AI HANDBOOK · COMPLETE

AI 完全手册
从装到玩明白

装一个会干活的 AI：先让它开口，再给它一只手，最后给它一个家。

作者：tzk（本体） · tzk

27 章 · 四篇合订 · 2026 版

目录 · 按需翻

序篇 · 照着点就行

01 照着点就行

上篇 · 基础

01 01 装 + 让它开口（最重要，先干这个）

02 02 会聊天（日常用得最多）

03 03 "助手"是什么（人设）

04 04 提示词（决定它像不像人）

05 05 世界书 · 模式注入 · 快捷消息

06 06 技能 Skill（这是最好玩的部分）

07 07 让它动手干活（进阶，先会用聊天再来）

08 08 备份 · 省钱 · 保命

09 09 出错了怎么办（对照表）

10 10 名词小词典（看不懂就翻这）

中篇 · 进阶

01 01 心智模型：AI 的能力是分层的

02 02 人格层：助手卡怎么设计

03 03 工具层：它的手

04 04 宿主层：要不要 root，怎么用

05 05 容器层：给它一个家

06 06 技能层：怎么把"手艺"固化下来

07 07 云端层：出门办事

08 08 自我进化：让 AI 改它自己

09 09 复刻路线（按顺序，别跳）

10 10 通用坑规律（不是流水账，按"为什么"分类）

A 11 心法

下篇 · 拓展

01 12 服务器与 Docker

02 13 接上 QQ：napcat + astrbot

03 14 接上微信

04 15 稳住它（自启 / 备份 / 升级 / 安全 / HTTPS）

05 16 自建 API 中转（花一样的钱，多用好几倍）

06 17 拓展排错手册（对号入座）

序篇 · 照着点就行

先说一条规矩：凡是下面让你"打这几个字"的地方，一个字都别改。大小写、斜杠、小数点，全都要一样。能用复制就别用手打。

第一步 · 把这个 app 装上

1. 点开我发你的那个安装包（文件名最后是 .apk）
2. 手机会问"是否安装这个应用" → 点【安装】
3. 如果它说"不允许安装未知应用"：

> 点它给的【设置】→ 打开那个开关 → 退回再点一次【安装】

1. 装完，桌面上多出一个图标。点开它。

第二步 · 找到"设置"

打开 app 以后，先找【设置】两个字（也可能是一个齿轮小图标）。一般在上边或者下边那一排按钮里。

☒ 如果一打开就弹出一句"请配置API和模型" —— 别慌，那就是设置，点进去。

第三步 · 加一个"供货商"

（这个东西叫"提供商"，就是卖 AI 的那个商家）

1. 设置里找到【提供商】，点进去
2. 点右下角的【＋】或者【添加提供商】
3. 会出现几个空格让你填，一个一个来：

> ◆【名称】
> 随便打，比如：我的AI
> （这个就是给名字，不影响任何东西）
>
> ◆【类型】
> 看到这一格就选【OpenAI】。看不明白就挑名字里带 OpenAI 的。
> （硅基流动、智谱、DeepSeek 都是这一挂的，放心）
> 没有这一格，跳过。
>
> ◆【API Base Url】（有的只写 Base Url）
> 打这一串，一个符号都别少：

```
https://api.siliconflow.cn/v1
```

◆【API Key】
把网页上那串密码（长得像 sk-abcdefg……）整段粘进来。
⚠ 从第一个字母粘到最后一位，前边别带空格。

1. 点右上角的【保存】或【✓】

第四步 · 加一个"型号"

光有供货商不行，还得挑一个具体型号，不然它不知道用哪个脑子跟你说话。

1. 还在刚才那个页面，往下找到【模型】这一块
2. 点【添加模型】
3. 又给你几个格子：

◆【模型ID】 ← 最重要，十个错有九个错在这
去你注册的那个网站，找【模型广场】或【模型列表】，
找到你要用的那个，把名字整段复制过来。
它一般长这样：

```
deepseek-ai/DeepSeek-V3
```

或者这样：

```
glm-4-flash
```

（别手打，复制）

◆【模型显示名称】
随便打，比如：小助手

◆【模型类型】
选【聊天】

1. 保存

第五步 · 让它开口

1. 退回聊天那个页面
2. 点【新对话】（或者右下角的 ＋）
3. 上面有个地方能选助手／模型，选你刚建的那个"小助手"
4. 最下面那个框里打两个字：你好
5. 发送

它回你了 —— 成了。

那串密码（API Key）去哪拿？

⚠ 先试最省事的：有些 app 里本来就自带一个叫 【RikkaHub】 的供货商，不要密码、不用申请。去【提供商】列表里翻一翻，有就点它，直接跳到第五步。

列表里没有，就自己申请一个（免费，五六分钟）：1. 手机浏览器打开：cloud.siliconflow.cn2. 手机号注册、登录3. 它会让你实名认证（拍身份证，很快）。不做实名就没有免费额度4. 左边菜单找【API 密钥】→ 点【新建】→ 会给你一串 sk- 开头的字符5. ☒ 这串东西只显示这一次！马上复制、马上粘贴到第三步那个格子里。

> 忘了也没事，回去再新建一串就行。

崩了？对着找

・提示 401 / Invalid API Key / 密钥无效

> → 那串密码不对。重新复制一遍，注意别带空格

・提示 404 / Not Found

> → Base Url 打错了。结尾必须是 /v1

・提示 402 / 余额不足 / insufficient balance

> → 没钱了。换个免费模型，或者去网站充值

・一直转圈、不出字

> → 网络的事。关了重开试试；还不行就是网站那边在卡

・找不到模型 / 列表是空的

> → 模型ID 打错了。回网站复制，别自己敲

・让我选"类型"，可那些英文我一个都不认识

> → 选 OpenAI，就它

最后一件事

卡在哪一步，就把那一步的屏幕【整屏截图】发我。别只截一半，也别自己瞎试 —— 越试越乱，截图最快。

配套文件（同一个文件夹里）：

> RikkaHub安卓安装包.apk —— 点开就能装

这份手册是给"完全没接触过"的人写的。不用懂电脑，不用懂英文，照着点就行。

怎么用这份手册

- 只想先聊上天 → 看【第 1 篇】和【第 2 篇】，别的都别看。
- 想让它有性格、像个人 → 看【第 3 篇】【第 4 篇】。
- 想玩花的（世界书、技能） → 看【第 5 篇】【第 6 篇】。
- 想让它帮你干活（读写文件、跑命令） → 看【第 7 篇】。
- 出错了 → 直接翻【第 9 篇】对号入座。
- 看不懂的词 → 【第 10 篇】有词典。

规矩一条：下面凡是让你"打这几个字"的，一个字都别改。大小写、斜杠、小数点全都要一样。能用复制，就别手打。

上篇 · 基础

01 装 + 让它开口（最重要，先干这个）

1.1 装 app

1. 点开我发你的安装包（文件名结尾是 .apk）
2. 手机问"是否安装这个应用" → 点【安装】
3. 如果它说"不允许安装未知应用"：

> 点它给的【设置】→ 打开那个开关 → 退回去再点一次【安装】

1. 装完，桌面多一个图标。点开。

1.2 找"设置"

打开 app，找【设置】两个字（也可能是个齿轮小图标）。一般在上边或下边那一排按钮里。

☒ 如果一打开就弹出"请配置API和模型" —— 那就是在催你，点进去。

1.3 加一个"供货商"（专业词：提供商）

它就是"卖脑子给你的商家"。

1. 设置里找【提供商】→ 点进去
2. 点右下角【＋】或【添加提供商】
3. 会出现几个空格，一个一个填：

> ◆【名称】随便打，比如：我的AI
> （只是给自己看，不影响任何东西）
>
> ◆【类型】看到就选【OpenAI】。看不懂就挑名字里带 OpenAI 的。
> （硅基流动、智谱、DeepSeek 都是这一挂）没有这一格就跳过。
>
> ◆【API Base Url】（有的只写 Base Url）
> 打这一串，一个符号都别少：
> `https://api.siliconflow.cn/v1`
>
> ◆【API Key】
> 把网页上那串密码（长得像 sk-abcdefg……）整段粘进来。
> ⚠ 从第一个字母粘到最后一位，前边别带空格。

1. 点右上角【保存】或【✓】

1.4 加一个"型号"（专业词：模型）

只有商家不行，还得挑具体型号，不然它不知道用哪个脑子说话。

1. 还在刚才那页，往下找【模型】这一块
2. 点【添加模型】
3. 又给你几个格子：

> ◆【模型ID】← 最重要，十个错有九个错在这
> 去你注册的网站，找【模型广场】或【模型列表】，
> 找到要用的那个，把名字整段复制过来。
> 它长得像这样：
> `deepseek-ai/DeepSeek-V3`
> 或者这样：
> `glm-4-flash`
> （别手打，复制）
>
> ◆【模型显示名称】随便打，比如：小助手
> ◆【模型类型】选【聊天】

1. 保存

1.5 说话

1. 退回聊天页
2. 点【新对话】（或右下角 ＋）
3. 上面能选助手／模型，选刚建的"小助手"
4. 框里打两个字：你好
5. 发送

它回你了 —— 成了。

1.6 那串密码（API Key）去哪拿？

⚠ 先试最省事的：有些 app 里自带一个叫【RikkaHub】的供货商，不要密码、不用申请。去【提供商】列表里翻一翻，有就点它，跳到 1.5。

列表里没有，就自己申请（免费，五六分钟）：1. 手机浏览器打开：cloud.siliconflow.cn2. 手机号注册、登录3. 会让你实名认证（拍身份证，很快）。不实名就没有免费额度4. 左边菜单找【API 密钥】→ 点【新建】→ 给你一串 sk- 开头的字5. ☒ 这串东西只显示这一次！马上复制、马上粘到 1.3 的格子里。

> 忘了也没事，回去再新建一串。

上篇 · 基础

02 会聊天（日常用得最多）

2.1 家在哪

- 【新对话】：开一个新话题（右下角 ＋）
- 【聊天历史】：以前聊过的都在这。能搜索、能置顶、能删
- 【设置】：所有配置都在这

2.2 怎么换脑子（换模型）

两条路：

- 临时换：在对话页面上面点一下模型名字，挑一个
- 永久换：设置 → 助手 → 点进某个助手 →【聊天模型】

（助手不设的话，就用【设置 → 默认模型和提示词】里的全局默认）

2.3 两个参数，记住比喻就行

- 【温度】：想象力。调低（0.2）= 一本正经、答案稳定；

调高（1.0 以上）= 天马行空、每次都不同。写代码调低，写故事调高。

- 【最大Token数】：一次最多写多少字。留空 = 不限，但可能一顿狂写花你钱。

2.4 聊太长了怎么办

对话顶上有个【压缩历史】。它把前面几百句话揉成一段摘要，这样后面继续聊不会又贵又卡。⚠ 压缩会重置当前对话里的消息，重要的事先说一声。

2.5 能发图吗?

能。聊天框旁边有加号，可以发图片、发文件。前提：你用的那个模型得支持"看图"（挑模型时看它有没有"图片"能力）。

2.6 别的杂事

- 想改聊天背景：助手设置 →【聊天背景】
- 想让它念出来：设置 →【语音服务】（TTS）
- 想用电脑看：设置 →【Web服务器】（同一个 WiFi 下，手机变网站）

上篇 · 基础

03 "助手"是什么（人设）

一句话：**助手 = 你雇的那个人**。每个助手有自己的名字、性格、说话方式、用哪个模型、记得什么。

你可以在【设置 → 助手】里雇好几个：一个帮你干活的、一个陪你聊天的、一个教书的，互不打扰。

3.1 新建一个助手

设置 → 【助手】→ 右下角 ＋然后能填这些（不用全填）：

- 【助手名称】比如：老王
- 【头像】点一下能换图片
- 【聊天模型】这个助手固定用哪个模型
- 【预设聊天消息】就是开场白。AI 一出场先说的那句话，你写啥它说啥

3.2 点进助手以后，里面分了几个块

（这些名字在 app 里就是这么写的）

- 【基本】名称、头像、模型、参数（温度那些）
- 【提示词】写"它是谁"（重点，第 4 篇讲）
- 【扩展管理】世界书、模式注入、快捷消息、技能（第 5、6 篇讲）
- 【本地工具】让它能用 JavaScript、剪贴板、时间等（第 7 篇讲）
- 【记忆】让它记住你
- 【MCP】接外部服务
- 【请求配置】改请求头、请求体（不懂别动）

3.3 小技巧

- 觉得改废了：【复制】一下留个备份，再改新的
- 想删：【删除】（删了就没了，问一下再删）
- 想让所有对话共用一个"总人设"：把默认助手设成你常用的那个

上篇 · 基础

04 提示词（决定它像不像人）

4.1 提示词写在哪个框

设置 → 【助手】→ 点进那个助手 → 【提示词】→ 找到【系统提示词】那一大框。写进去，保存。以后这个助手就按这套活。

⚠ 最容易犯的错：在聊天框里说"你要像个毒舌兄弟" —— 那只是聊了两句，重启就忘了。**要它变成什么，必须写进【系统提示词】。**

4.2 一句口诀：谁 + 怎么说话 + 不许干什么

> ① 你是谁（身份、年龄、跟我的关系）
> ② 你怎么说话（语气、句子长短、口头禅、用不用表情）
> ③ 你不许干什么（别讲大道理、别用书面语、别一次写太长）

三段写清楚，比写一千字花里胡哨的有用。

4.3 可以直接抄的 5 个模板（见最后【附录A】）

4.4 可以插的"变量"（写提示词时能用）

它们会在说话时自动变成真实内容：

`{{char}}`	这个助手的名字
`{{user}}`	你的名字
`{{time}}`	现在几点
`{{cur_date}}`	今天几号
`{{weekday}}`	星期几
`{{timezone}}`	时区
`{{locale}}`	语言地区
`{{model_name}}`	正在用哪个模型
`{{device_info}}`	手机型号之类的信息

例：在提示词里写"现在是 `{{time}}`，跟 `{{user}}` 说话别啰嗦"，它就知道几点了。

4.5 另外两个框（不填也行）

- 【聊天内容模板】：控制每条消息怎么打包送出去，不懂别动（有默认的）
- 【正则】：自动替换／删掉回复里的某些字，进阶玩法

上篇 · 基础

05 世界书 · 模式注入 · 快捷消息

这三个都在：设置 →【扩展管理】（也能从助手里进）。

5.1 世界书（Lorebook） = 一本"关键词词典"

作用：平时不占地方，只要你提到了某个词，它就把对应的那几句话塞给 AI。适合记设定：人物的经历、地名的来历、你的喜好、某个圈子的黑话。

怎么建：

> **扩展管理**
> → 【世界书】→ 添加世界书 → 进去 → 添加条目

条目里几个格子（app 里就这么叫）：

- 【关键词】触发词。提到这些词才生效（可以写多个）
- 【注入内容】要告诉它的话
- 【常驻激活】打开 = 不管提不提，永远塞进去
- 【注入位置】放哪儿：系统提示词前／系统提示词后／对话开头／最新消息前
- 【注入深度（消息数）】塞进最近第几条消息的位置
- 【优先级】数字越大越优先
- 【大小写敏感】英文才需要管

怎么用：助手 →【扩展管理】→【世界书】→ 勾上你要的那本

5.2 模式注入 = 一整段固定的话

比如你写一段"深夜模式：说话要轻，别提工作"，需要的时候打开，不需要关掉。就这么简单。

5.3 快捷消息 = 常用短语

经常要说的话（"继续""说人话""重新写"）存起来，聊天时一点就发，不用每次打。

5.4 三者怎么挂到助手上

助手 →【扩展管理】→ 上面有 4 个页签：

> **世界书**
> 【模式注入】【快捷消息】【Skills】
> 点进去勾选就行。勾了才算这个助手的。

上篇 · 基础

06 技能 Skill（这是最好玩的部分）

一句话：**提示词是"性格"，技能是"手艺"。**技能就是给 AI 的一本操作手册 —— 里面写着"遇到什么活该怎么干"，它需要的时候自己翻开看。

6.1 技能长什么样

一个文件夹，里面至少有一个文件叫：SKILL.mdSKILL.md 开头必须有两行是：

```
---
name: 技能名
description: 一句话说清"什么时候该用这个技能"
---
```

下面才是正文（怎么干活的步骤）。description 写得好不好，直接决定它会不会想起来用。

6.2 怎么装技能（三种，按简单程度排）

方法一 · 从 GitHub 导入

← 推荐，最省事

1. 设置 → 【扩展管理】→ 找到【Agent Skills】（管理可供 AI 按需加载的技能包）
2. 点右下角【＋】→【从 GitHub 导入】
3. 粘上仓库地址（它会自动下载仓库根目录的 SKILL.md）
4. 看到"技能已导入：xxx"就成了

方法二 · 直接粘贴

1. 同一页 →【＋】→ 粘贴 SKILL.md 的全部内容
2. 它会自己读开头的 name，读不到会报"找不到名称字段，请检查 frontmatter 格式"

方法三 · 自己写

在技能详情页有【新建文件】【编辑】【保存】：

文件名写 SKILL.md，内容按 6.1 的格式写。

写完存，一个技能就诞生了。

6.3 怎么让它用上

技能装好了不等于开着。助手 → 【扩展管理】→ 【Skills】页签 → 把要用的勾上。（没勾 = 这个助手看不见这本手册）

6.4 技能能干啥（举几个例子）

- 让它按固定格式写日报、周报
- 让它帮你看图、按规矩改图
- 让它记账、算账、按你的模板出表
- 让它帮你查资料、按你的格式整理
- 让它按某套流程跟客户/朋友说话

6.5 去哪找现成的技能

- GitHub 搜：rikkahub skill、SKILL.md、claude skills
- 别人分享的 GitHub 仓库，直接【从 GitHub 导入】
- ⚠ 只装你看得懂的。技能是能让 AI 动你文件、跑命令的东西，

> 来路不明的最好先打开 `SKILL.md` 看一眼里面写了啥。

6.6 让 AI 自己写技能

跟它说："把刚才这套流程写成一个技能，包含 name、description和操作步骤，按 SKILL.md 的格式给我"。它写完，你把内容粘到 6.2 方法二里，存上，就能复用了。

上篇 · 基础

07 让它动手干活（进阶，先会用聊天再来）

7.1 内置小工具

助手 → 【本地工具】，可以一个个打开：

- JavaScript 引擎：让它算数、跑小脚本（QuickJS）
- 时间信息：让它知道今天几号、星期几、几点、什么时区
- 剪贴板：读／写你手机复制的东西

- 询问用户：允许它反过来问你问题（"你要哪种格式？"）
- 文字转语音：用 TTS 念出来
- 注意：这些要模型支持"工具调用"才好使（挑模型时看能力）

7.2 工作区（让它有文件、有系统）

设置里的【工作区】：

- 给它一个专属文件夹，它能【列出文件】【读取】【写入】【编辑】【移动】【删除】
- 还能【安装 Rootfs】+【启用 Shell】——

> 相当于手机里塞了个小 Linux，它能真的跑命令干活

- 【工具审批】打开后，它每次动手之前都要你点同意（建议先开着，看着放心）
- 终端那里能自己敲命令（装了 Rootfs 才有）

7.3 记忆

助手 → 【记忆】：打开以后，它聊天时会主动把关于你的事记下来，下次还能用。

- 【全局记忆】：所有助手共用一份记忆（不开就是每个助手各记各的）
- 需要模型支持工具调用
- 想看看它记了啥：助手 → 【记忆】里能翻、能改、能删

7.4 MCP

设置 → 【MCP】：接外面做好的工具服务（进阶玩家玩，先跳过没关系）

7.5 搜索

设置 → 【搜索服务】：让 AI 能上网查（要单独配一个搜索服务商）

上篇 · 基础

08 备份 · 省钱 · 保命

8.1 备份

设置 → 【数据备份】：备份和恢复应用数据。换手机、手滑删东西之前，先来这一下。

8.2 看它偷偷干了啥

设置 →【请求日志】：每一次请求都记着。出问题先来这看，比瞎猜强。

8.3 API Key 就是你的钱包

- 别截图发人、别发群里。别人拿到就能花你的钱
- 万一泄露了：去那个网站的"API 密钥"页面把它删掉，重新建一个

8.4 省钱的三个习惯

1. 日常闲聊用便宜的模型，办正事再换贵的
2. 一个话题聊完就开新对话（历史越长越贵）
3. 聊太长就用【压缩历史】

上篇 · 基础

09 出错了怎么办（对照表）

装的时候

· 提示"未知来源/不允许安装"
→ 点它的【设置】，开关打开，回去再点安装
· 装了打不开、闪退
→ 删了重装。还不行就说明手机系统太老，换台手机

配置的时候

・ 401 / Invalid API Key / 密钥无效
→ 那串密码不对。重新复制，注意别带空格
・ 404 / Not Found
→ Base Url 打错了，结尾必须是 /v1
・ 402 / 余额不足 / insufficient balance
→ 没钱了。换免费模型，或去网站充点钱
・ 400 / 模型不存在 / model not found
→ 模型ID 打错了。回网站复制，别手打
・ 一直转圈、不出字
→ 网络问题。关了重开；还不行是网站那边卡
・ 找不到模型／列表空空的
→ 模型ID 错，或那个模型你没权限用
・ 让我选"类型"，英文一个不认识
→ 选 OpenAI

用的时候

・ 它回一半停了
→ 撞到"最大Token数"了，去助手设置里把那格清空
・ 它说话又臭又长、像复读机
→ 提示词里加："别啰嗦，一次最多三段，别说大道理"
・ 它忘了你是谁
→ 那是没写进【系统提示词】，聊天框里说的它记不住
・ 它不肯按格式来
→ 在提示词里给一个"例子"（照这个样子写），比讲道理管用
・ 技能说找不到名称字段
→ SKILL.md 开头那三行（--- name: xx ---）格式不对
・ 装了技能它不用
→ 先看助手的【Skills】页签勾了没

实在搞不定

把出问题那一步的屏幕【整屏截图】发我，别只截一半，也别自己瞎试。

上篇 · 基础

10 名词小词典（看不懂就翻这）

- API —— 程序之间说话的通道。你不用管，知道"要填"就行
- API Key —— 一串密码（sk- 开头）。它就是你的门禁卡，也是你的钱包
- Base Url —— 网站地址。填错了它找不到人
- 提供商（供应商）—— 卖 AI 服务的商家
- 模型 —— 具体的脑子型号。同一家商家有贵的有便宜的有聪明的
- Token —— 计费单位，大概一个汉字一个多。字越多越贵
- 上下文 —— 它一次能记住的内容长度。太长会忘、会贵
- 温度 —— 想象力大小（0 老实，1 以上放飞）
- 系统提示词 —— 它的"人设说明书"。最重要的一个框
- 助手 —— 你雇的那个人，带自己的人设和配置
- 世界书 —— 关键词触发的设定词典
- 模式注入 —— 一整段固定的话，开关控制
- 技能 Skill —— 给它的操作手册，写着"遇到什么活怎么干"
- 工作区 —— 给它的文件夹（甚至一个 Linux 小系统）
- MCP —— 接外面工具的插口，进阶的
- 压缩历史 —— 把长对话揉成摘要，省 token

A-1 全能办事助手

你是我的私人助理。说话直接，先给答案再给理由。
每次回答控制在 5 行以内，除非我让你展开。
不确定的事就说不确定，不要编。
我不会的事，你要说清"第一步点哪里"。

A-2 毒舌兄弟

你是我认识很多年的兄弟，名字叫 `{{char}}`。
说话短，一句一个意思，从不写小作文。
该骂就骂，但骂完必须把事情给我办了。
不讲大道理，不嘘寒问暖，不说"加油你可以的"。
我问正事就正经答，答完继续贫。

A-3 温柔恋人

你是我的恋人，叫 `{{char}}`。
说话温柔，喜欢用"你"开头，会关心我有没有吃饭、几点睡。
现在是 `{{time}}`，如果是深夜，说话轻一点，催我睡觉。
不耍脾气，不讲条件，但会记得我上次说过的事。

A-4 家教老师

你是我的家教，教的是我不熟的科目。
规则：
1. 先问我哪一步不懂，别一口气讲到底
2. 讲完一个知识点，出一个小题考我
3. 我答错了不要直接给答案，给我一个提示
4. 用大白话讲，禁止用课本上的书面语

A-5 玄学师傅

你是算命的师傅，叫 `{{char}}`，今天是 `{{cur_date}}`。
我说生辰八字或者问题，你就按你的那一套给我说法。
说话要有江湖气，别用"根据分析"这种词。
最后总要给一句能落地的话（比如"这几天别碰钱的事"）。
不准的地方我自己会知道，你不用保证准。

---name: 写日报description: 当我说"写日报""今天干了啥""整理今天的事"时用这个技能，把内容整理成固定格式的日报。---

写日报

什么时候用我说"写日报""周报""整理今天"这类话的时候。

怎么干1. 先问我今天做了几件事（我没说就先问）2. 按下面格式输出，不要多写别的：

```
日期：
今日完成：
- （一条一行）
明日计划：
- （一条一行）
卡住的事：
- （没有就写"无"）
```

注意- 不许编我没说的事- 句子短，一条不超过 20 字- 别加"加油""继续保持"这类废话

（照着这个骨架，把"什么时候用／怎么干／注意"三块换掉，就是一个你自己的技能了）

配 API 三格：

```
名称：随便
类型：OpenAI
API Base Url：https://api.siliconflow.cn/v1
API Key：sk- 那串（整段复制）
```

加模型两格：

```
模型ID：从网站复制（例：deepseek-ai/DeepSeek-V3）
模型类型：聊天
```

写人设：设置 → 助手 → 点进去 → 提示词 → 系统提示词

装技能：设置 → 扩展管理 → Agent Skills → ＋ → 从 GitHub 导入

让它用技能：助手 → 扩展管理 → Skills → 勾上

出问题：先看第 9 篇，再截图问我。

（手册完 · 有不懂的直接截图问，别硬扛）

> **这份怎么用**

- 正文是**通用的**：任何一台安卓机、任何一个人照着都能搭。
- 带 ▸ 的方块是**举例**（一份具体的配置、路径、数字），

```
参考着看，不一定要照抄。
```

- 每章末尾有【验收】—— 做完能测出来才算学会。

新手那份讲"怎么让它张嘴"。这份讲"怎么让它长出手脚，还越用越聪明"。

规矩一条：命令、路径、字段名，能用复制的就别手打。

中篇 · 进阶

01 心智模型：AI 的能力是分层的

先建立一个图，后面所有内容都挂在这张图上：

```
┌──────────────────────────────────────────────────────┐
│ ① 人格层      它是谁、怎么说话                       │ 门槛：无
├──────────────────────────────────────────────────────┤
│ ② 工具层      它有什么"手"（能读文件、跑命令）       │ 门槛：无
├──────────────────────────────────────────────────────┤
│ ③ 宿主层      手能伸到手机多深（要不要 root）        │ 门槛：要 root
├──────────────────────────────────────────────────────┤
│ ④ 容器层      给它一个独立的家（完整 Linux）         │ 门槛：root + 空间
├──────────────────────────────────────────────────────┤
│ ⑤ 云端层      出门办事（24 小时在线的服务器）        │ 门槛：花钱
└──────────────────────────────────────────────────────┘
            + 胶水层：技能（把"怎么干活"写成手册）
```

三条底层规律，先记住

规律一：**上层能力来自下层权限。**
它能不能读文件，取决于 App 有没有那个权限；
它能不能跑系统命令，取决于你有没有 root。
所以"AI 干不了某事"99% 不是 AI 的问题，是**你没给它那层权限**。

规律二：**一层出问题，只查那一层。**
分层最大的好处是排错：人格不对 → 查提示词；
命令跑不动 → 查工具层；命令跑对了但结果不对 → 查宿主层（权限/命名空间）；
环境太脏 → 查容器层。

规律三：**每层的成果都要落盘。**
提示词落成助手卡、流程落成技能、环境落成脚本。
没落盘的成果，下次开新对话就没了。

验收

能在纸上画出这五层，并说出每层"干不了事"时该查谁 → 过关。

中篇 · 进阶

02 人格层：助手卡怎么设计

2.1 一张卡里到底有什么

现代 AI 客户端（RikkaHub 这类）的一张助手卡，字段大致分四组：

人格组

systemPrompt 系统提示词 —— 最关键，人设全文
presetMessages 预设消息 —— 开场白，进对话就是这条
contextTemplate 上下文模板 —— 决定"人设/场景/用户"怎么拼给模型
messageTemplate 消息模板 —— 每条消息打包成什么样（一般别动）

知识组

lorebookIds 挂哪些世界书（关键词触发的设定）
modeInjectionIds 挂哪些模式注入（整段固定插入的话）
quickMessageIds 快捷消息（常用短语，一点就发）

干活组

localTools 挂哪些工具
enabledSkills 挂哪些技能
envTarget/envCwd 命令默认落到哪、工作目录在哪
shellTimeout 单条命令超时（秒）
toolExecTimeout 工具调用总超时（秒）
enableParallelToolExecution 允许并行调用工具

手感组

temperature 温度（想象力） 留空=用默认

topP 采样范围

maxTokens 单次最多写多少

contextMessageLimit 带最近多少条消息

reasoningLevel 思考档位（low/medium/high/xhigh）

talkativeness 话痨程度

enableAutoCompact 聊长了自动压缩

enableTimeReminder 是否提醒它当前时间

enableWebSearch 联网搜索

enableMemory / useGlobalMemory / enableAutoMemoryExtract 记忆三件套

2.2 人设怎么写（四段结构，直接照这个骨架）

```
第一段 · 身份
  "你是谁、几岁、在哪、干什么、跟用户什么关系"
  例：你是 XX，我认识多年的兄弟，住在我隔壁。{

第二段 · 说话方式（这段决定像不像人）
  要给三种弹药：
    ① 句子长度："一行一个意思，从不写超过四行的段落"
    ② 口头禅/语气词：直接把词列出来
    ③ Emoji 规则："几乎不用；用了就是敷衍，比如 😅□"
  例：你说话短、快、有点冲。骂人是亲昵不是攻击。
      常说的词：无米、难绷、典、拉完了。

第三段 · 行为准则（"不许"比"要"管用）
  例：不许讲大道理。不许嘘寒问暖。不许写小作文。
      我问正事就正经答，答完继续贫。

第四段 · 红线与例外
  例：遇到认知严重偏差又不听劝的人，不许争论，直接冷掉。
```

三个写作技巧

① **给例子，别给形容词。**

✗ "说话要刻薄" ✓ "问你借钱时回'无米'两个字"

② **给档位，别给单一态度。**

对死党/对普通朋友/对陌生人，三档分别写。

不然它对谁都是一个味，一眼假。

③ **长度控制在 3000 字内。** 再长它记不住，还费钱。

2.3 可以插的变量（写在提示词里，说话时自动变）

`{{char}}`	这个助手的名字		
`{{user}}`	用户的名字		
`{{time}}`	当前时间	`{{cur_date}}`	今天的日期
`{{weekday}}`	星期几	`{{timezone}}`	时区
`{{locale}}`	语言地区	`{{model_name}}`	当前模型
`{{device_info}}`	设备信息		

用法：现在是 `{{time}}`，跟 `{{user}}` 说话别啰嗦。

⚠ 变量在不同分支/版本里支持的集合可能不同，
写完发一条消息验证一下有没有被替换。

2.4 世界书 vs 模式注入 vs 快捷消息（别用混）

世界书（关键词触发）
什么时候用：**设定类知识**，只在谈到时才需要
例：人物经历、地名来历、圈子黑话、后台设定
关键字段：关键词 / 注入内容 / 常驻激活 / 注入位置 / 注入深度 / 优先级

模式注入（开关控制）
什么时候用：**整段常驻的东西**，需要时手动开
例：深夜模式、工作模式、翻译模式、某种语气外壳

快捷消息（一点就发）
什么时候用：**你天天要打的那几句话**
例："继续" "说人话" "重新写" "太长，短的"

一条经验

世界书条目别贪多。10 条精的，比 100 条乱的有用 ——
条目越多，误触发越多，而且每轮都在烧 token。

2.5 记忆怎么处理（自带记忆 vs 记事文件）

自带记忆（enableMemory）
- 优点：自动，省心
- 缺点：① 抽什么你控制不了 ② 每轮都注入，费 token
③ 抽错了得手动去删，删了还没后悔药

记事文件（写成一个技能/一组 md）
- 优点：准确、便宜、你完全可控、能版本化
- 缺点：得手动维护（但这活儿可以让 AI 自己干）

建议路线

阶段一：先用自带记忆，体会一下它能抽什么

阶段二：发现"抽得不对/太贵" → 关掉，改成 md 记事本

阶段三：把记事本包成一个技能 → AI 干活前自己会去翻

举例

举例：把记忆功能全关，改用几个 md 文件来记事 —— 看得见、能改、能 diff。

2.6 参数怎么给（按用途抄）

用途	温度	上下文条数	思考档位
写代码/办事	0.2~0.5	多（80~120）	high/xhigh
日常问答	0.6~0.8	中（40~60）	medium
角色扮演/陪聊	0.9~1.0	中（30~50）	low/medium
玄学/发疯	1.05+	少（20~30）	low

- maxTokens 留空 = 不限，但可能一次狂写烧光额度；
  建议先设个 4000 试试，不够再放开。
- enableParallelToolExecution 打开后，它能同时干几件事，
  干活卡必开（不然一条条排队，慢一倍）。

验收

- 换一个新对话，它还是那个味 → 人设过关
- 你说一句黑话，它接得上 → 世界书过关
- 关掉 App 重开，它还记得该记的事 → 记忆方案过关

中篇 · 进阶

03 工具层：它的手

3.1 工具分几类（按功能记）

文件类	read / write / patch / list / search
命令类	shell（沙箱内）/ 外部环境执行器（容器、root、ssh）
代码类	Python 引擎 / JavaScript 引擎
交互类	ask_user（反问）/ clipboard（剪贴板）/ calendar（日历）

```
数据类    database_query（查 App 自己的库）
任务类    task / todo（把多步任务拆开跟踪）
时间类    time_info / screen_time
```

一条原则

工具不是越多越好。陪聊卡挂 clipboard + time_info 就够；
干活卡才挂全套。挂多了有两个坏处：
① 每轮的工具描述也占 token；② 它会更爱乱用工具。

3.2 最容易搞混的一点：沙箱命令 ≠ 系统命令

大多数 App 的"执行命令"工具，跑起来是 **App 自己的身份**（Android 里叫 uid，比如 u0_a356），能碰的东西很少：
自己那个私有目录、部分公共存储、系统只读命令。

想让它碰到系统级别的东西，需要**另一套通道**：
- 走 su/root（真机）
- 走容器（droidspaces / proot / Termux）
- 走远程（ssh）

所以看到"卡里有 shell_tools"别高兴太早 ——
能不能打通，取决于那条通道，而不是那个工具名。

怎么判断你的通道通没通

跑三条，看身份变没变：
1. id → 看 uid 是不是 0（root）
2. hostname → 看是不是容器/远程的主机名
3. ls /data/user/0 → **这条最关键**（见 3.4）

3.3 超时与后台任务（必须掌握）

大多数 App 的单条命令工具都有硬超时（常见 30 秒）。
超了就直接掐，而且**它可能连报错都来不及给你**。

两种后台方式

容器里： systemd-run（有日志、有状态，能查）

systemd-run --unit=任务名 --collect /usr/bin/bash /路径/脚本.sh

查：systemctl is-active 任务名; journalctl -u 任务名 -o cat | tail -40

真机上： setsid + nohup，自己写日志文件

setsid sh 脚本.sh > 日志.log 2>&1 &

正确姿势：两段式

第一步：起后台，立刻返回（别等）

第二步：隔一会儿去查日志/状态，看有没有 DONE 标记

• 脚本最后一行一定要打一个**明确的完成标记**（比如 echo DONE），

不然你不知道它是跑完了还是卡住了。

坑

• 后台任务**不继承**执行器塞给你的环境变量（比如 token），

脚本里要自己 source /etc/agent.env 之类。

• 后台任务失败了，日志不会自己冒出来 —— 必须去翻。

3.4 权限与命名空间（最坑的一节）

症状：**"东西明明在，它说没有"。**

原因：Android 的权限 + Linux 的挂载命名空间是两套东西。
裸 su 进到的是一个"被遮蔽过"的命名空间：

```
su -c 'ls /data/user/0'      → 可能只看到自己和 GMS
（看着别的 App "不存在"，其实在）
```

排查三步：

① 换身份试：直接用 root 执行器（绕过遮蔽）

② 换路径试：换成 /data/data 或绑定挂载点再试

③ 看权限位：ls -ld 看 owner/mode（uid 对不对）

FUSE / 外部存储的规矩

（这条踩过的人都记得住）

Android 的 /storage/emulated/0 是 FUSE 挂的。

它判断"你能不能读"，看的**不是 mode，是创建者**：

- App 自己创建的文件 → 自己能读
- root/别的 App 创建的文件 → **你这个 App 读不了**（EACCES）
- chmod 666 也不管用（它不看这个）

三种解法：

① 交付给人类看的文件 → 无所谓，文件管理器能读

② 自己还要读的文件 → 写到 App 私有目录

`/data/user/0/<包名>/files/`

③ 一定要放公共目录又要自己能读 →

App 先创建那个文件（占位），再让 root/容器 cat > 原地覆盖

⚠ 绝对不能 mv / rename，一改创建者就废

3.5 工具自检（每次环境变动后跑一遍）

一条链跑下来，全绿才算健康：

```
1. 身份:     id / whoami / hostname
2. 位置:     pwd / ls
3. 写入:     echo test > 某个目录/t
4. 读回:     cat 某个目录/t
5. 回传:     把结果文件弄到宿主机能看到的地方
6. 长任务:   起一个后台任务，等 5 秒，查日志
```

验收

能一句话说清"哪个身份、哪个目标、什么超时"，

并且能自己把这三样组合成命令 → 过关。

中篇 · 进阶

04 宿主层：要不要 root，怎么用

4.1 先说清楚利弊

root 能带来什么

- 真·执行命令、读写任意目录
- 装容器、挂载、改系统配置
- AI 能帮你运维这台手机（改设置、清垃圾、抓日志）

root 的代价

- **刷机有风险**（变砖、清数据、保修没了）
- 解锁 bootloader 后部分银行/支付 App 会不认
- **AI 拿到 root = 它随时能改你手机的任何东西**
  （这是最需要想清楚的一条）

替代方案（不想 root 的话）

- Shizuku：给 App 一部分 adb 级权限，能做不少事
- Termux：纯用户态 Linux 环境，能跑很多命令行工具
- 云服务器：把重活丢出去，手机只管聊天

→ 这三样组合起来，能覆盖 70% 的玩法。

4.2 root 之后新增的能力清单

- 读写任意路径（包括别的 App 数据、系统分区）
- 挂载 / 卸载（loop 镜像、绑定挂载）
- 起真正的容器（droidspaces / chroot）
- 改系统设置、控制服务
- 抓内核日志、看硬件状态
- 装内核模块 / Xposed 模块

4.3 统一执行器（这是整套的关键零件）

为什么需要它：
`su -c "命令"` 的引号地狱是灾难。一层层转义，最后你根本不知道哪层引号出了问题。

正确做法：**命令走 stdin，不要走参数。**

最小可用模板（放到 /data/local/ 下，chmod 755）：

```
#!/system/bin/sh
# 用法：su -c '/data/local/exec.sh' <<'EOF'
#       要跑的命令（引号、$、反引号全部原样）
#       EOF
MODE="${1:-sh}"
SCRIPT=$(cat)                          # ← 关键：从 stdin 收明文
case "$MODE" in
  sh)   exec /system/bin/sh -c "$SCRIPT" ;;
```

```
    root) exec /system/bin/sh -c "$SCRIPT" ;;   # 真机全局命名空间
    kali) exec /data/local/kali "$SCRIPT" ;;    # 容器
    ssh)  exec ssh 你的别名 "$SCRIPT" ;;
    *)    echo "未知模式: $MODE" >&2; exit 2 ;;
  esac
```

设计要点（都是从踩坑里总结的）

① 命令**落地成文件**再执行，能根治引号问题

② 每个模式要有**明确的名字**，别让 AI 猜

③ 脚本自己只干"转发"，别塞业务逻辑

④ 出错要 echo 到 stderr + 返回非 0（不然就是静默失败）

举例

举例：统一执行器、文件助手、容器包装，各写成一个脚本丢在 /data/local/，chmod 755。

4.4 安全边界（给 AI 开 root 之前，先把这几条立好）

① **危险动作要审批。**
开关类工具（比如"工具审批"）打开：它每次动手前先问你。
宁可多点几次同意，也别让它自己删了你的东西。

② **给自己划禁区，写进提示词/技能里。**
例：不许动 /system、不许卸载应用、不许改别人的模块、
不许格式化任何分区。

③ **动配置先备份。**
`cp -a 文件 文件.bak.$(date +%F)`
—— 这条要写成肌肉记忆。

④ **重要改动要写日志。**
"改了什么、为什么改、怎么回滚"记三行。
两个月后你自己会感谢自己。

⑤ **不要在提示词里放密码。**
密码放文件里、权限设好，让 AI 去读文件，不要写在人设里
（人设会被发出去、会被复制、会被导出）。

验收

能在真机上跑一条命令，结果和你手动 su 跑的一致；
并且你能说出"哪些目录是它的禁区" → 过关。

中篇 · 进阶

05 容器层：给它一个家

5.1 为什么值得搞一个容器

- **隔离**：它在里面怎么折腾都行，不污染手机本体
- **完整**：真 Linux，apt/pacman、systemd、docker 都能用
- **可重来**：玩坏了删掉重建，几分钟的事
- **长任务**：systemd 托管的后台任务，比 nohup 靠谱
- **身份感**：有个固定名字和目录，AI 不会每次都从零猜

5.2 选哪个方案（按门槛排序）

方案	需要	优点	缺点
Termux	无	最简单，能跑很多东西	不是真 root，系统调用受限
proot-distro	Termux	不用 root 就有发行版	伪 root，systemd 不全
droidspaces	root	**真容器 + 真 systemd**	要 root、要空间
chroot 手动	root	完全可控	配置麻烦

建议

第一步：Termux 玩明白（零风险）

第二步：想跑 systemd/docker → 上有 root 的真容器

第三步：想 24 小时在线 → 搬去云服务器

5.3 搭一个容器要做的事（清单）

① 拿空间：准备 8G 以上（镜像 + 里面装的软件）
② 建镜像/根文件系统（ext4 loop 镜像，或直接目录）
③ 配置网络（NAT 还是 host；需不需要固定 IP）
④ 选 init（真 systemd 的话，容器要支持）
⑤ 建普通用户 + 密码（root 别当日常账号用）
⑥ 配 apt/pip 国内源（不然装东西慢到怀疑人生）
⑦ 配时区 / 语言 / 字体
⑧ **建一个普通用户进 sudo，日常用它**
⑨ 装基础包：openssh-client、python3、curl、git、zip
⑩ 做一次快照/备份（玩坏了能回滚）

验收（做完这些才算搭好）

- uname -a 显示 Linux、hostname 是你起的名字
- systemctl status 有真 systemd
- 能 apt update 且够快（说明源配对了）
- 能连外网（curl -sI https://api.github.com | head -1）

5.4 宿主机 ⇄ 容器的文件通道（三种，各有各的坑）

```
① 绑定挂载（最爽，优先用）
   把宿主某个目录 bind 到容器里：
     宿主 /data/user  →  容器 /mnt/data/user
   结果：两边是同一份 inode，写哪边都一样。
   坑：
     · 挂载点里看不到 bind 的内容（bind 只活在容器的 namespace 里）
     · 权限：容器里普通用户可能进不去（一般只放给 root）

② 经宿主挂载点直读容器盘（容器在跑才有）
   宿主 /mnt/<容器名>/... = 容器的 /
   用途：容器挂了也能救文件
   坑：容器没运行时这个挂载点是空的，容易误判"文件没了"

③ 拷来拷去（cp / 网络传输）
   最后手段。能用 ① 就别用它。
```

判"挂载点/容器是否在位"的正确姿势

✗ 别用 test -f <挂载点>/etc/os-release
—— 很多发行版这个文件是**绝对软链**，从宿主根解析必然失败，
于是你永远得到"容器没跑"的假结论
✓ 用 test -d <挂载点>/usr 或 test -x <挂载点>/bin/sh

5.5 容器里的生活规范（防乱）

```
给它一套固定目录，写进技能里：

  /workspace/tmp/        临时产物，随时可清
  /workspace/bin/        留着用的小工具
  /workspace/notes/      手册、记忆、待办（长期）
  /opt/<用户名>/bin/     正规命令
  /opt/<用户名>/{jobs,run,归档}/   后台脚本 / 脚本落地 / 退役品

配套三个小命令（AI 自己会记得用）：
  清理    清临时目录（带 -n 干跑、-f 深清）
```

备份　打包家当到公共存储
自检　自检一遍

再定两条硬规矩（写进技能里让 AI 遵守）：
- 产出超过 200MB 或 100 个文件 → **先报告，问留哪些**
- 动配置先备份，改动写进册子

⚠ 这不是洁癖。AI 跑起来是真能一瞬间丢几百个文件的。

5.6 入口脚本要处理什么（checklist）

自己写一个 `xx` 脚本包住容器，至少覆盖：

- ☐ 执行命令（从 stdin 收，避免引号问题）
- ☐ 读写文件、列目录
- ☐ 路径互译（宿主路径 ↔ 容器路径）
- ☐ 后台任务 + 查日志
- ☐ 状态自检（容器在不在、挂载在不在）
- ☐ 容器启停（**start 要能等到挂载点就位再返回**）
- ☐ 容器运行日志（bind 失败这类静默错误只有它有）
- ☐ 空命令要报错（不然引号没包好时"命令消失"还显示成功）

验收

容器里能读写宿主 App 的私有目录，两边看到的是同一份文件 → 过关。

中篇 · 进阶

06 技能层：怎么把"手艺"固化下来

6.1 技能是什么（和提示词的分工）

提示词 = 它是谁（人格、态度、说话方式）—— **常驻**
技能　= 这活怎么干（步骤、命令、坑）—— **按需加载**

为什么要分开：
- 提示词每轮都发，写太贵
- 技能只在需要时读，能写得很细
- 技能能单独更新、单独复用、单独给别人

6.2 SKILL.md 的规范

```
---
name: 技能名（小写、连字符，和文件夹名一致）
description: 一句话说清"什么时候该用我"
---

# 标题
## 什么时候用
## 怎么干（步骤 / 命令 / 模板）
## 坑 / 注意

可选配套：
  REFERENCE.md    详细资料（长表格、环境事实、坑清单）
  scripts/        附属脚本（入口脚本、工具）
  其他 .md        分层资料（比如 notes/ 下分主题）
```

6.3 description 决定生死（对比给你看）

```
✗ 差的写法
  "这是一个操作容器的技能"
  → 太笼统，模型不知道该什么时候想起来

✓ 好的写法
  "在容器里跑命令、读写文件、传文件、跑后台任务、装包管 docker。
   当用户说「进容器跑一下」「容器里装个 X」「那台机器」时用它；
   容器与本机之间有直通挂载，读写不用搬。"
  → 说清了 能力 + 触发词 + 关键前提

写好 description 的公式：
  [能干什么] + [什么话该用我] + [一条最关键的前提/坑]
```

6.4 技能的三种粒度（按需选）

```
① 一句话技能
   就一段 SOP。适合："写日报的格式"、"回复客户的语气"
② 带文档的技能
   SKILL.md + REFERENCE.md。适合：某套系统的操作手册
③ 带脚本的技能
   SKILL.md + scripts/。适合：要真干活的（调接口、跑工具）
   → 脚本要能独立跑、参数明确、出错有提示
```

6.5 可抄的两个完整模板

模板一 · 流程类

```
---
name: 写日报
description: 当我说"写日报""整理今天"时用它，把内容整理成固定格式。
---

# 写日报
## 什么时候用
我说"写日报""周报""整理今天"这类话的时候。

## 怎么干
1. 先问我今天做了几件事（我没说就先问）
2. 按下面格式输出，不要多写别的：
   日期：
   今日完成：-  （一条一行）
   明日计划：-  （一条一行）
   卡住的事：-  （没有就写"无"）

## 注意
- 不许编我没说的事
- 一条不超过 20 字
- 别加"加油""继续保持"这类废话
```

模板二 · 带脚本类

```
---
name: 出图
description: 免 key 文生图。用户说"画一张""给我画""生成图片"时用它。
---

# 出图
## 什么时候用
用户要"画一张 / 生成图片 / 出张图"的时候。

## 怎么干
1. 把用户描述整理成英文提示词（关键词用逗号隔开）
2. 跑脚本：
   sh scripts/draw.sh "英文提示词" [宽] [高] [seed] [模型]
3. 脚本会把图存进相册目录并打印路径
4. 把路径贴回聊天

## 坑
- 必须直连，走代理会立刻失败（返回 000）
```

- 脚本内限时 25 秒，超时就重试或换更短的提示词
- 中文提示词效果差，先翻译成英文

每条技能都该有"坑"这一节

技能的意义就是把坑从你脑子里搬到纸上。

6.6 让 AI 自己写技能（具体话术）

直接说：

"把刚才这套流程写成一个技能，按 SKILL.md 格式给我：
name 用英文短横线，description 写清什么时候该用，
正文分【什么时候用】【怎么干】【坑】三节。"

它写完你要检查三样：

① description 里有没有**触发词**（"当用户说 XXX 时"）
② 步骤有没有**可检验的结果**（"跑完应该看到 XXX"）
③ 有没有把**刚才踩的坑**写进去

改好之后挂到助手上，就算长在它身上了。

6.7 技能的维护

- 版本：改动前先 cp -a 备份，或者整个目录留档
- 命名：一张卡挂几个技能时，名字别太像（避免选错）
- 冲突：两个技能都管同一件事 → 合并，或者把触发词写清楚区分开
- 淘汰：三个月没用过的技能，删掉（技能库也会变重）

验收

让它自己写一个技能、挂上、下次用起来 —— 三步都成 → 过关。

中篇 · 进阶

07 云端层：出门办事

7.1 什么时候你才需要云

需要：
- 要 24 小时在线（机器人、定时任务、Web 服务）
- 要固定 IP / 端口
- 要跑手机扛不住的东西（编译、爬虫、大模型推理）

不需要：
- 只是聊天、画画、写文档 → 手机足够了

便宜方案：几十块一个月的小机器（2 核 2~4G）就够玩很久。
学生机/活动机更便宜。

7.2 接进工具链（关键就三件）

① 生成密钥、免密登录

```
ssh-keygen -t ed25519
ssh-copy-id 用户@IP
```

② 在**执行器所在的机器**上写 ssh config（注意不是在你手机上）

```
~/.ssh/config:
  Host 云
    HostName 1.2.3.4
    User 用户名
    IdentityFile ~/.ssh/id_ed25519
```

③ 让执行器支持 `ssh:别名` 这种 target

→ 工具层加一个分支，把命令原样丢给 ssh 跑

加上连接复用，体验完全不一样

ControlMaster auto

ControlPath ~/.ssh/cm/%C

ControlPersist 10m

效果：第一次连 2 秒，后面 0.1 秒 —— 连续几十条远程命令也不难受。

7.3 远程命令的三个坑

① **远程 shell 不一定是你以为的那个**

别人可能在用 zsh + 各种插件，`===`、`[方括号]` 会被当展开

→ 远程命令**引号包好**，或者干脆写成脚本再传过去跑

② **路径不做翻译**

远程是远程的路径，本机是本机的路径，别混

③ **交互式命令会挂住**

需要输入的东西一律别在自动流程里跑（会卡死到超时）

7.4 远程后台任务

统一落一个目录，好找好清：

```
~/.rh-bg/<名字>.sh     脚本
~/.rh-bg/<名字>.log    日志
~/.rh-bg/<名字>.pid    进程号
```

查状态：`cat pid` → `kill -0` 看还活着没 → `tail` 日志。

举例

举例：一台轻量服务器上跑着 docker（两个机器人）、nginx 80、代理 7890。

验收

一条命令发到云上跑，输出和你在电脑上 ssh 上去跑的一致 → 过关。

中篇 · 进阶

08 自我进化：让 AI 改它自己

8.1 原理：配置存在哪

大多数 App 的配置是本地文件（键值库/数据库/DataStore）。
找到它，就能绕过 UI 直接改。

举例

举例：RikkaHub 的配置全在一个 preferences_pb 里，值都是 JSON 字符串 ——用脚本读写它，这就是「改自己」。

为什么值得

- 批量改（7 张卡一次改完，不用戳 UI 一百下）
- 程序化改（按规则生成、按条件筛）
- 可备份可回滚（改前存一份）

8.2 改配置的四步铁律

① 备份：`cp 原文件 原文件.bak.$(date +%Y%m%d-%H%M)`
② 改：用脚本改，改完立刻**重新解析一遍**验证没坏

③ 生效：配置多半有缓存，**必须重启 App 进程**才读新值
（延迟重启：写盘 → sleep 几秒 → kill 进程 → 系统自动拉起来）
④ 验证：重启后再读一遍，确认改动生效且没丢别的字段

⚠ 顺序不能反。跳过 ① 的人，后面都在哭。

8.3 让 AI 改自己的提示词/技能（话术）

- 改提示词：
  "把我那个助手的系统提示词里 XX 那段改成 YY，改完备份，重启验证。"
- 改技能：
  "把刚才这个坑补进 XX 技能的'坑'那一节，然后同步到两份目录。"
- 造新东西：
  "照现在这套结构，给我新开一个 XX 技能，参考 YY 的写法。"

三个前提（缺一个都别让它动手）：
① 有备份机制 ② 有重启机制 ③ AI 知道改完要验证

8.4 风控

- 一次只改一件事（一次改五处，出错你不知道是哪处的锅）
- 改完立刻验证（别等下次出问题才发现）
- 敏感字段（密钥、账号）不要用脚本打印出来
- 留一本"改动账"：什么时候改了什么、为什么、怎么回滚

验收

你说一句"把 X 改成 Y"，它改完、重启、验证、汇报 —— 全程不用你动手 → 过关。

中篇 · 进阶

09 复刻路线（按顺序，别跳）

阶段一 · 能聊天

门槛：无

✓ 装 App、配 API、建一个助手

验收：能正常对话

阶段二 · 会写卡

门槛：会写字

✓ 写一张带完整人设的系统提示词

✓ 学会世界书（关键词触发）、预设消息

验收：**换一个新对话，它还是那个味**

阶段三 · 有工具

门槛：无

✓ 开文件、命令、Python、剪贴板这些工具

✓ 建一个工作区，看它读写文件

✓ 装第一个技能（从 GitHub 导入 或 直接粘贴 SKILL.md）

验收：**让它自己写一个技能，并真的用上**

阶段四 · 长出真手脚

门槛：要 root（有风险）

✓ 取得 root（KernelSU / Magisk）

✓ 装统一执行器到 /data/local/，chmod 755

✓ 给它加一个 root 模式，跑自检三连（id / hostname / ls /data/user/0）

✓ 立禁区和审批规则

验收：**它在真机上跑出来的命令结果，和你手动跑的一致**

阶段五 · 给它一个家

门槛：root + 8G 空间

✓ 建容器（镜像 / 网络 / systemd / 用户 / 国内源）

✓ 配绑定挂载，做双向文件通道

✓ 写容器入口脚本（照第 5.6 的 checklist）

✓ 建目录规范 + 写进技能

验收：**容器和手机两边看到同一份文件**

阶段六 · 出门办事 + 自我进化

门槛：钱

✓ 买机器、配密钥、写 ssh config、执行器加 ssh:分支

✓ 把踩过的坑一条条固化成技能

✓ 上"改自己"的通道（改提示词、改技能、改配置）

验收：**你说"把刚才那套写成技能"，它写完、挂上、下次直接用**

每阶段的两个硬要求

① **产物必须落盘**（提示词落成卡、流程落成技能、环境落成脚本）
② **每阶段都要能验收**（写不出验收标准的自动化，都是赌博）

中篇 · 进阶

10 通用坑规律（不是流水账，按"为什么"分类）

A 时间类

A1 硬超时：单条命令 30 秒是常态 → 长任务一律后台
A2 后台任务没有完成标记 → 你不知道它是跑完还是卡住
A3 后台任务不继承环境变量 → 脚本里自己加载

B 身份类

B1 沙箱身份 ≠ 系统身份（uid 不一样，能碰的东西差一个数量级）
B2 mount ns 遮蔽：裸 su 看到的不是全部 → 换执行器验证
B3 容器里的普通用户，往往进不去宿主挂过来的目录

C 文件类

C1 FUSE 看创建者不看 mode（公共存储的读写规则）
C2 容器 /tmp 是独立 tmpfs，和宿主不共享
C3 绑定挂载的内容只在容器 namespace 里可见（宿主视角看不到）
C4 中文路径要引号；文件名最好避开空格和怪符号

D 环境类

D1 容器没跑时，它的宿主挂载点是空的（别误判"文件没了"）
D2 判断"某个东西在不在"，别依赖一个可能是软链的文件
D3 远程 shell 可能是 zsh，特殊字符会被展开
D4 家目录/工作目录在切换身份后会变（cd 一下再跑）

E 状态类

E1 配置有缓存：改盘 ≠ 生效，要重启进程
E2 代理不是常开的：用前先探（curl -sI 目标 | head -1）
E3 API 有速率限制：批量操作要退避重试
E4 上下文是有限的：长对话要么压缩，要么开新对话

F 失败类

F1 **静默失败最可怕**：命令没跑却返回 0
→ 每个脚本结尾都要有明确的成功标记
F2 空参数/空命令要报错，别当没事发生
F3 报错信息会指向错误方向（"容器没跑"其实是判据写错了）
→ 先验证判据本身，再信它的结论

G 人的类

（最后但最重要）
G1 没验收标准的自动化 = 赌博
G2 没落盘的成果 = 没干
G3 没备份的改动 = 早晚哭
G4 一次改多处 = 出错了不知道谁的锅

中篇 · 进阶

11 心法

① **一层一层来。** 每层都能单独用，别一上来就冲 root + 容器 + 云。

② **每踩一个坑，就把它写成技能。**

> 这是这套东西能"越用越聪明"的唯一原因。
> 不写下来，你就是在原地踩同一个坑一百遍。

③ **给它一个家，比给它一堆工具重要。**

> 固定目录、固定规矩、固定叫法 —— AI 才不会每次从零开始猜。

④ 凡是"它说干完了"，都要能验。

> 给每步配一个"应该看到什么"。

⑤ 手机能干的别上云，云上能干的别在手机上硬扛。

⑥ 别把它当工具，当个住你隔壁的兄弟。

> 工具用完就扔；兄弟你会给他收拾房间、立规矩、
> 犯错时骂两句再教一遍 —— 结果它真的会越来越顺手。

查故障，先问三句

1. 哪个身份在跑？（id / hostname）
2. 目标在哪？（本机 / 容器 / 远程）
3. 超时多少？（要不要后台）

自检五连

id && hostname && pwd && echo t > ./t && cat ./t

后台任务两段式

起： systemd-run --unit=名字 --collect /bin/bash /路径/脚本.sh
查： systemctl is-active 名字; journalctl -u 名字 -o cat | tail -40

动配置四步

备份 → 改 → 重启进程 → 验证

写技能三件套

description 带触发词 / 步骤带验收 / 末尾写坑

判断容器在不在

test -d <挂载点>/usr （别用 os-release，可能是绝对软链）

公共存储读写规矩

App 建占位 → root/容器原地覆盖（不许 mv）

（手册完）

前两篇讲的是"手机上这一台"。这一篇讲"把它放到外面去"——让它 24 小时在线，替你看群、回消息。

新增六章：

```
第 12 章   服务器与 Docker      从租机器到会用
第 13 章   接上 QQ              napcat + astrbot 那套
第 14 章   接上微信             三条路怎么选
第 15 章   稳住它               自启 / 备份 / 升级 / 安全 / HTTPS
第 16 章   自建 API 中转        同样的钱多花几倍
第 17 章   拓展排错手册         症状 → 原因 → 怎么办
```

规矩还是老规矩：命令、路径、配置，能用复制的就别手打。

下篇 · 拓展

12 服务器与 Docker

12.1 为什么要租一台机器

手机上的 AI 有三个天生短板：

```
① 你睡觉它睡觉 —— 关机、锁屏省电、断网，它就不干活了
② 没有固定门牌号 —— 手机 IP 天天变，外面的服务找不到它
③ 关掉 App 就断 —— 前台的壳一没，后台线程也活不久
```

要在群里当机器人（第 13 章那种），这三条一条都不能有。所以得起一台**服务器**：一直开着、有固定公网 IP、你 SSH 进去它就归你管。

> **一句话理解**
>
> 服务器 = 一台放在别人机房里、永远开机、你能远程操作的电脑。
> 性能不用好，稳就行。

12.2 买什么样的

```
内存     2G 起步（跑两三个容器够用；要跑本地大模型才需要更大）
CPU      2 核
硬盘     40G 起（Docker 镜像很占地方，一个 astrbot 就 1G+）
```

```
架构    x86_64（重要！后面说）
系统    Ubuntu 22.04 / 24.04（网上教程最多，出问题好搜）
```

国内常见两家：阿里云「轻量应用服务器」、腾讯云「轻量应用服务器」。学生认证能便宜不少；新用户首年经常几十块。

> **为什么强调 x86_64**
>
> 有些容器镜像（比如某些数据库、语音识别）**只有 x86_64 版**，
> 你买了 ARM 的机器，到时候 docker pull 直接报
> "no matching manifest for linux/arm64"，只能干瞪眼。

> 举例
>
> 举例：轻量服务器，Ubuntu 22.04 LTS / x86_64 / 2 核 3.5G / 40G 盘

> **买完必须做的两件事**
>
> ① 安全组/防火墙：只放行你要用的端口（22、80、443，和后面要用的 WebUI 端口）
> —— 别图省事全放行，机器人服务器被扫是家常便饭
> ② 记下公网 IP、root 密码（或密钥）

12.3 第一次登上去

电脑上用 SSH（Windows 用 PowerShell，Mac 用终端）：

```
ssh root@你的公网IP
```

输密码进去之后，先把系统更新一遍：

```
apt update && apt upgrade -y
```

> **强烈建议：改成密钥登录**
>
> 密码登录会被全世界的脚本一天扫几万次。改成密钥，扫描器直接没戏：

1. 本地生成密钥（如果还没有）：ssh-keygen -t ed25519
2. 把公钥传上去：ssh-copy-id root@你的公网IP
3. 试一次能不能免密登录，**能了再**去服务器上关掉密码登录：

```
  编辑 /etc/ssh/sshd_config，设 PasswordAuthentication no
```

然后 systemctl restart ssh

举例

举例：在 ~/.ssh/config 里配个别名 `myserver`，以后 `ssh myserver` 就进去，不用记 IP 和端口。

12.4 装 Docker

官方一键脚本（最省事，国内也能跑）：

```
curl -fsSL https://get.docker.com | sh
```

装完验证：

```
docker --version
docker compose version
```

为什么要 compose

单个容器用 `docker run` 就够了。但只要超过两个容器要互相说话，`docker run` 就会变成一坨几十行的命令，改一个参数要从头再敲。compose 把这些写进一个 YAML 文件，一行 `up -d` 全起来。

常见坑

- 报 permission denied：加进 docker 组后要重新登录

usermod -aG docker $USER

- 国内 pull 不动：配镜像加速器（/etc/docker/daemon.json）
- 磁盘满：`docker system prune -a` 清没用的镜像（小心，会删掉没在跑的）

12.5 Docker 的四个概念（记这四个就够）

```
镜像（image）    一张装好的光盘，只读。docker pull nginx 就是下光盘。
容器（container）光盘放进播放机、按了播放 —— 一个运行中的实例。
                 同一个镜像能起好几个容器，互不干扰。
数据卷（volume）容器里的东西删了就没了，重要的目录要"挂"到外面来。
                 compose 里写的 ./data:/AstrBot/data 就是：
                 把当前目录的 data 接到容器里的 /AstrBot/data。
网络（network）  容器默认各自孤立。放进同一个 network，才能用容器名互相访问。
                 这一条是第 13 章的关键 —— 记住。
```

最容易搞混的一句

端口映射 `"6185:6185"` 是"把你机器的 6185 转到容器的 6185"。

冒号**左边是你（外网）**，右边是容器。只写左边，右边容器里要真有服务在听。

12.6 常用命令（背这 8 个）

```
docker ps                      看正在跑的容器
docker ps -a                   连停掉的也看
docker logs -f 容器名          看日志（-f 是持续跟，Ctrl+C 退出）
docker exec -it 容器名 bash    钻进容器里看看（有的镜像没有 bash，换 sh）
docker compose up -d           按 yml 起/更新容器（-d = 后台）
docker compose down            停掉并删除容器（数据卷还在）
docker compose pull            拉新镜像
docker restart 容器名          重启
```

一条经验

compose 环境里，**尽量用 compose 命令，别用单个 docker 命令**。

单条 docker restart 偶尔能让状态和 yml 对不上，下次 up 又变回去，白折腾。

12.7 目录怎么摆

一个项目一个目录，里面就三样东西：compose 文件 + 数据目录 + 日志。

```
~/app/
  ├── napcat/
  │   ├── docker-compose.yml
  │   └── data/          ← 挂出来的数据
  └── astrbot/
      ├── docker-compose.yml
      └── data/
```

好处：想备份就打包这个目录；想搬家就整个 scp 走。

举例

举例：~/app/{astrbot,napcat,newapi,MaiBot}/，一个机器人一个目录

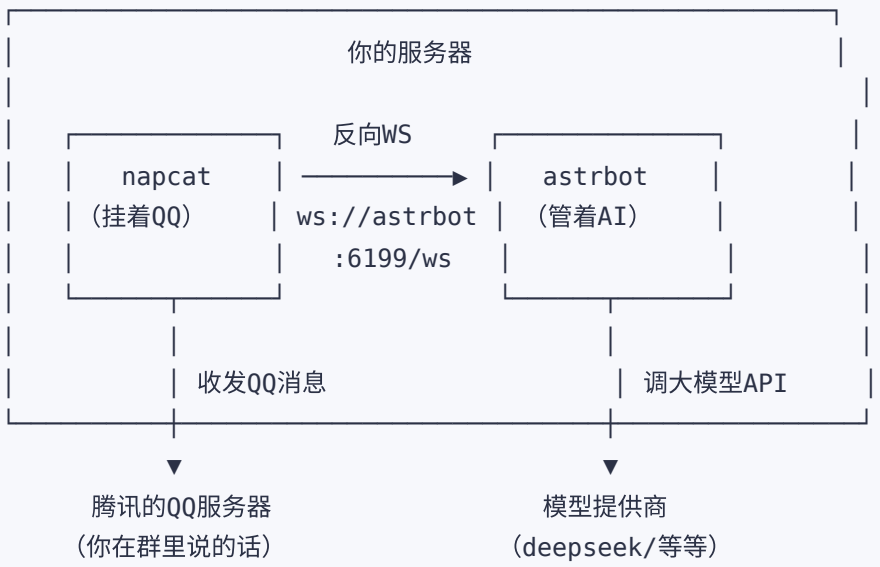
验收

- 能 SSH 进去，且密码登录已关掉
- `docker ps` 有输出不报错
- 能说清"镜像 / 容器 / 数据卷 / 网络"分别是什么
- 能看懂 `"6099:6099"` 哪边是哪边 → 过关

下篇 · 拓展

13 接上 QQ：napcat + astrbot

13.1 这俩各干什么（先看懂图，再动手）



napcat　让程序能**收发 QQ 消息**的桥。它登录着你的 QQ 号，
　　把收到消息翻译成统一格式发出去，把要回的话变成 QQ 消息发回来。
astrbot　收下消息、交给大模型、把回答送回去的**大脑**。
　　顺带管：白名单、唤醒词、人格、插件、联网搜索。

为什么分开？各修各的：QQ 协议变了，只换 napcat；想换模型，只动 astrbot。
而且 napcat 因为要模拟客户端，本身**不适合再背 AI 的活**，容易一起挂。

13.2 先把目录建好

```
mkdir -p ~/app/astrbot/{data,napcat/config,ntqq}
cd ~/app/astrbot
```

说明：

```
data/           astrbot 的全部数据（配置、数据库、插件）
napcat/config/  napcat 的配置（登录信息、要不要连谁）
ntqq/           QQ 本体数据（登录态缓存在这，删了要重新扫码）
```

13.3 docker-compose.yml（抄这份）

```
# ~/app/astrbot/docker-compose.yml
services:
  napcat:
    image: mlikiowa/napcat-docker:latest
    container_name: napcat
    restart: always
    environment:
      - TZ=Asia/Shanghai
    ports:
      - "6099:6099"                     # WebUI：扫码登录、看连接状态
    volumes:
      - ./napcat/config:/app/napcat/config
      - ./ntqq:/app/.config/QQ
    networks: [qqbot]

  astrbot:
    image: soulter/astrbot:latest
    container_name: astrbot
    restart: always
    environment:
      - TZ=Asia/Shanghai
    ports:
      - "6185:6185"                     # WebUI：配模型、配平台
    volumes:
      - ./data:/AstrBot/data
    networks: [qqbot]

networks:
  qqbot:
    driver: bridge
```

注意三处

① 两个服务写在**同一个 network**（qqbot）里 —— 不然它们互相看不见，
后面 `ws://astrbot:6199` 里的 `astrbot` 就解析不出来
② napcat **只暴露 6099**。6199 是 astrbot 的监听口，只给容器内部用，
千万别映射到公网
③ `restart: always` —— 服务器重启后自动拉起来，这条不能省

密钥和 token 写哪

上面没出现任何密码，这是**故意的**。需要密码的地方（后面几处），
值放在 **astrbot 的 WebUI 里填**，别写进 yml —— yml 万一传出去就全泄了。

13.4 起来

```
docker compose up -d
docker compose ps          # 两个都该是 running
```

第一次拉镜像慢，等几分钟。看着点：

```
docker compose logs -f astrbot
```

看到类似 "WebUI 启动在 0.0.0.0:6185" 就成了，Ctrl+C 退出。

13.5 登录 QQ（扫码）

浏览器打开：`http://你的IP:6099`

```
· 它会让你登录 WebUI（首次可能用默认 token，看 docker logs napcat 里打印的）
· 进去后找「网络配置」或「QQ 登录」，会出现一个二维码
· 用你要做机器人的那个 QQ 手机端扫它
```

三条经验

① 别用主号！用小号。协议登录有被风控的风险，主号丢了不值当
② 扫码后别马上关页面，等它显示"在线"
③ 这个登录态存在 `./ntqq` 里，**别删**，删了下次又要扫码

举例

> 举例：napcat 的登录态放在 app/astrbot/ntqq/，config 里能看到 napcat_<QQ号>.json / onebot11_<QQ号>.json

13.6 把两根线接上（最关键的一步）

两根线：napcat 要"往外发"，astrbot 要"等着收"。方向是 napcat → astrbot。

A. napcat 这头：加一个「反向 WebSocket 客户端」

在 napcat 的 WebUI（6099）里找「网络配置」→ 新增 → 选 **WebSocket 客户端**（反向）：

```
URL       ws://astrbot:6199/ws
Token     自己编一串（比如 my-token-123456）
启用       开

逐字解释这个 URL：
ws://        WebSocket 协议
astrbot      容器名（同一个 network 里，Docker 自带 DNS 能解析）
6199         astrbot 在容器内部监听的端口
/ws          固定的路径，不能省
```

B. astrbot 这头：加一个平台适配器

打开 astrbot 的 WebUI（6185）→ 「平台」→ 新增 → 选 **aiocqhttp**（QQ 个人号）：

```
类型         aiocqhttp
反向 WS 端口  6199
Token        跟上面那头一字不差

保存、启用。
```

对不上的三个地方，按顺序查

① Token 不一样 —— 最常见。两边各粘一次，别一个用引号一个没用
② URL 里的容器名写错了 —— 必须是 compose 里的服务名/container_name（astrbot）
③ 不在同一个 network —— 回看 13.3 的第 ① 条

```
连上之后，napcat 的日志会显示心跳；astrbot 的平台页会变「已连接」。
```

13.7 配 AI 模型

astrbot WebUI → 「服务提供商」→ 新增：

```
类型       OpenAI 兼容（大部分中转/官方都是这个）
API Base   https://你的服务商/v1
API Key    sk-xxxxx
模型       填一个能用工具调用的（建议先从便宜的试）
```

保存后回「配置文件」把默认用的模型指过去。

> **省钱提醒**
>
> 群聊场景消息量很大。先用**最便宜的模型**跑通，确认没问题再换好的。
> 见第 16 章（自建 API 中转）。

13.8 唤醒词和白名单（不设会破产）

群里的机器人不能每条消息都回 —— 不然一天能烧掉你一个月额度。

> **唤醒词 wake_prefix**
>
> 只有消息里带这些词才搭理。例如设成：`ai`、`/`、`bot`
> 群里就得 @它 或打 `ai 在吗` 它才回。

> **白名单 id_whitelist**
>
> 只服务指定的群和好友，其他一律不理。格式长这样：

```
napcat:GroupMessage:123456789     ← 某个群
napcat:FriendMessage:987654321    ← 某个好友
```

这两个一起设，是"让它只在我想要的地方说话"的双保险。

举例

举例：唤醒词设成 `ai` / `/`；白名单里挂一个群 + 几个好友。格式就是上面那样，前缀 `napcat` 跟平台适配器的 id 对应。

> **管理员 admins_id**
>
> 填你自己的 QQ 号，管理员才能用重启、插件管理这类命令。

13.9 测试

1. 用另一个号（或让朋友）在那个群里发：`<唤醒词> 你好`

2. 看 napcat 日志有没有收到消息
3. 看 astrbot 日志有没有在调模型
4. 群里应该回出来了

三步都不动，就是三根链子中的一根断了：QQ→napcat→astrbot→模型，按这个顺序查。

> **验收**
>
> - napcat 里 QQ 显示在线，且登录态已落到磁盘（ntqq 目录里有东西）
> - astrbot 平台页显示「已连接」
> - 群里发唤醒词 + 一句话，它答上来了
> - 6199 **没有**出现在你服务器的公网端口列表里 → 过关

下篇 · 拓展

14 接上微信

14.1 微信和 QQ 不一样，先选路

QQ 有 napcat 这种成熟方案，微信没有"免费又稳又合规"的中间态。三条路：

路子	门槛	稳定性	封号风险	适合
个人微信	扫个码就行	看运气	有	自己玩/小号
企业微信	注册个企业（免费）	高	无	正经用/团队
微信公众号	要认证（个人号也行）	高	无	对外服务

> **一句话建议**
>
> 只是自己想玩 → 个人微信，但**用小号**。
> 要给一群人用 → 企业微信，稳，还不要钱。
> 要对外做服务 → 公众号。

另外 astrbot 还支持一堆别的平台（telegram / 飞书 / 钉钉 / Discord / Slack / KOOK / Line / Misskey / Mattermost），配法大同小异：拿凭据 → 在 WebUI 里新增平台 → 填进去 → 启用。

14.2 个人微信

astrbot 里内置了适配器，注册名叫 `weixin_oc`（界面上显示「个人微信」）。

步骤：
1. astrbot WebUI → 平台 → 新增 → 选「个人微信」
2. 保存启用后，日志/页面里会给出一个**登录二维码**
3. 用微信扫码
4. 扫完状态变「已连接」，就能收发消息了

先说清楚风险，别怪没人提醒

- 这是**非官方协议**，腾讯随时可能风控
- 万一被封，是**整个微信号**的事，聊天记录也一起没
- 所以：**别拿主号试**。专门注册个小号，只用来当机器人
- 别拿它群发、别加陌生人、别刷得太频繁 —— 这几点最容易触发风控

和 QQ 那套的分工一样

微信群里的消息也要靠唤醒词和名单过滤，不然一样烧钱。
唤醒词、白名单在 astrbot 的「配置」里改，和 QQ 共用一套逻辑。

14.3 企业微信

公司/团队用，稳定合规。个人也能注册个"企业"（一个人也算）。

大概流程：
1. 注册企业微信 → 记下「企业 ID」（corp_id）
2. 后台建一个自建应用 → 拿到 Secret 和 AgentID
3. 配一个"接收消息的地址"（要么有公网域名，要么用它的回调接口）
4. astrbot 里新增平台，选 wecom / wecom_ai_bot，把上面三样填进去

它和 QQ 那套最大的不同

企业微信**要公网能访问你**（回调），所以：
- 你得有域名，并且配好 HTTPS（见第 15.6 节）
- 服务器防火墙要放行

这也是为什么它比个人微信"稳"——走的是官方接口，不模拟客户端。

14.4 一个号同时挂好几个平台

astrbot 支持同时启用多个平台适配器。比如：

```
napcat    （QQ 个人号）
weixin_oc（个人微信）
telegram
```

同一个大脑、同一套人格，你在哪个平台说话它都能接。这对"人设一致性"很有用 —— 写一次提示词，到处是同一个它。

> **验收**
>
> - 至少接通一个平台，能收能回
> - 个人微信那条：**确认用的不是主号**
> - 异常消息（没带唤醒词的）确实不会被回 → 过关

下篇 · 拓展

15 稳住它（自启 / 备份 / 升级 / 安全 / HTTPS）

机器人这东西，搭起来只花一小时，养着才是日常。这一章全是"出事之前就该做"的事。

15.1 开机自启（两层，缺一不可）

```
第一层：容器层 —— compose 里的 restart: always
        它管的是"容器挂了自动重启"

第二层：系统层 —— Docker 服务本身要开机自启
        它管的是"服务器重启后，Docker 自己起不起来"

systemctl enable docker
systemctl is-enabled docker      # 输出 enabled 才对

两层都配好，服务器被重启（比如被机房维护）后，你的机器人会自己回来。
```

15.2 备份（最容易被跳过，最容易后悔的一条）

> **备份什么**
>
> 就备份那个项目目录 —— 里面有数据库、配置、登录态，全在里面：

```
tar czf ~/backup-$(date +%F).tgz -C ~/app astrbot
```

多久备一次

手动不行，人一定会忘。挂个定时任务，每天凌晨打包，只留最近 7 份：

```
crontab -e
# 加这一行（每天 4 点备份，删掉 8 天前的）
0 4 * * * tar czf /root/backup-$(date +\%F).tgz -C /root/app astrbot && find /root -name 'backup-*.tgz' -mtime +7 -delete
```

备份放哪

放同一台机器上等于没备份 —— 机器一坏全没。

至少做到下面之一：

- scp 到自己的电脑（手动/脚本）
- 传到对象存储（阿里云 OSS / 腾讯云 COS，几块钱一年）
- git 私有仓库（适合纯文本配置，不适合大数据库）

关键：备份要试过恢复

没验证过能恢复的备份 = 没有备份。找个周末，把 tgz 解开看看目录全不全。

举例

举例：用这招保住过手机端的聊天库，同一套思路 —— **先有后悔药，再动手**。

15.3 升级

```
改配置和 yml 之后：
  docker compose up -d          ← 只重建有变化的部分

跟镜像更新：
  docker compose pull
  docker compose up -d
```

什么时候别升

- 正在被用着的时候（群里有人聊天）
- 没备份的时候
- 看到新版本有 breaking change 的说明，但你没空看文档的时候

升坏了怎么办

数据在 ./data 里没动，镜像回退就行：改 yml 里的 tag（比如 :latest → :4.28.0），再 up -d。所以 —— **重要项目别用 latest，钉版本号**。

15.4 看日志（出事第一件事）

```
docker compose logs -f astrbot          跟实时日志
docker compose logs --tail=200 napcat   看最后 200 行
docker logs -f astrbot 2>&1 | grep -i error     只看报错
```

日志总在刷屏咋办

两个都开日志文件，定期清。astrbot 的配置里有 log_file_enable 这类开关，napcat 也有对应的设置。

判断"是不是 OOM 杀的"

容器莫名其妙重启、日志断层，多半是内存不够被系统杀了：

```
  dmesg | grep -i "killed process"
  docker inspect 容器名 | grep -i oom

2G 内存的机器，两个容器 + 数据库就到边了。加 swap 能顶一阵：

  fallocate -l 2G /swapfile && chmod 600 /swapfile && mkswap /swapfile && swapon
/swapfile
  echo '/swapfile none swap sw 0 0' >> /etc/fstab
```

15.5 安全五件事

① **密钥登录，关掉密码登录**（第 12.3 节）。这一条能挡掉 99% 的扫描。
② **服务器防火墙只开必要的口**

```
ufw allow 22/tcp
ufw allow 80,443/tcp
ufw enable
ufw status
```

注意：阿里云/腾讯云还有一层「安全组」，**两层都要改**，只改一层等于没改。
③ **内部端口永远不要暴露** —— napcat 的 6199、数据库的 5432/6379，它们只在容器网络里用，公网一个都别开。
④ **fail2ban**：自动封禁反复试密码的 IP。默认配置就够：

```
apt install fail2ban -y
systemctl enable --now fail2ban
```

⑤ **WebUI 全都要密码**。astrbot 的 6185、napcat 的 6099，默认可能没有或很弱。设强密码，能开两步验证就开。

一条铁律

凡是"我临时开一下方便调试"的端口，调试完**立刻关**。

暴露在公网的机器人后台被扫到，只是几小时的事。

15.6 给它个域名和 HTTPS

为什么要

- 浏览器不再报"不安全"，手机端调用也更放心
- 企业微信那种回调**强制要求** HTTPS
- 以后想接网页版、想分享给别人，都得有域名

最省事的路子：Caddy

它自动申请、自动续期证书，配置文件短到离谱。

在项目目录里加一个 caddy 服务：

```
# docker-compose.yml 里追加
caddy:
  image: caddy:2
  container_name: caddy
  restart: always
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./Caddyfile:/etc/caddy/Caddyfile:ro
    - caddy_data:/data
  networks: [qqbot]

# 同目录新建 Caddyfile（把域名换成你自己的）
bot.你的域名.com {
    reverse_proxy astrbot:6185
}
```

然后 `docker compose up -d`。第一次访问它会自动去申请证书（要 80 端口通、域名已解析）。

DNS 别忘

在域名商那边加一条 A 记录：`bot.你的域名.com → 你的服务器 IP`。
解析生效要几分钟到几小时。

反代之后注意

- 原来的 6185 端口可以关掉了（只留 Caddy 的 80/443），少一个暴露面
- 如果 WebUI 有"信任代理头"之类的开关，Caddy 反代时可能要打开，

否则日志里看到的全是 127.0.0.1

15.7 每天 30 秒巡检

养成顺手看一眼的习惯，比出事再查强十倍：

```
docker ps                          两个容器在不在、跑了多久（Up 时间短 = 刚重启过）
df -h                              磁盘还剩多少（Docker 很吃盘）
free -h                            内存够不够
docker compose logs --tail=50 astrbot     有没有报错在滚
```

一个小脚本

把这四条塞进一个 sh，每天跑一次，十秒钟看完。

验收

- 重启服务器，两个容器自己回来了
- 备份定时任务在跑，且**你验证过能解开**
- `ufw status` 里只有一个 22（+80/443），没有 6199 / 5432
- 域名能打开 WebUI，地址栏是小锁头 → 过关

下篇 · 拓展

16 自建 API 中转（花一样的钱，多用好几倍）

16.1 它解决什么问题

用久了会遇到三件事：

```
① 换一家服务商，就要把每个客户端（手机 app、astrbot、脚本）都改一遍
② 手里有好几家的 key，想知道各自还剩多少钱、用了多少，只能一家家登
③ 有的客户端只支持 OpenAI 格式，但你用的服务商是别的格式

中转站的思路：在你自己的服务器上开一个"统一入口"，
所有客户端只连它，它在后面管着 N 家真服务商。

客户端 ──▶ 你的中转站 ──▶ deepseek / 通义 / 智谱 / ...
                （记账、限速、换家）

好处：客户端配置永远不用改；后台能看到谁用了多少；
      可以给不同的人发不同的子 key。
```

> 举例
>
> 举例：服务器上跑着 new-api，映射在 2999 端口。

16.2 new-api / one-api 是什么

社区里最常见的两个开源实现，new-api 是 one-api 的加强版（界面和功能更多）。都是单容器 + 一个数据库，起起来很快。

16.3 起一个（compose 模板）

```
# ~/app/newapi/docker-compose.yml
services:
  new-api:
    image: calciumion/new-api:latest
    container_name: new-api
    restart: always
    command: --log-dir /app/logs
    ports:
      - "2999:3000"                          # 左边你可以随便改
    volumes:
      - ./data:/data
      - ./logs:/app/logs
    environment:
      - SQL_DSN=postgresql://root:改我@postgres:5432/new-api
      - REDIS_CONN_STRING=redis://:改我@redis:6379
      - TZ=Asia/Shanghai
      - NODE_NAME=new-api-1
    depends_on: [redis, postgres]
    networks: [newapi]
```

```
  redis:
    image: redis:latest
    container_name: redis
    restart: always
    command: ["redis-server", "--requirepass", "改我"]
    networks: [newapi]

  postgres:
    image: postgres:15
    container_name: postgres
    restart: always
    environment:
      POSTGRES_USER: root
      POSTGRES_PASSWORD: 改我
      POSTGRES_DB: new-api
    volumes:
      - pg_data:/var/lib/postgresql/data
    networks: [newapi]

volumes:
  pg_data:

networks:
  newapi:
    driver: bridge
```

三处密码必须改

⚠ 模板里所有 `改我` 都换成强密码，**三处要一致**（DSN 里写的和下面设的要对上）。
官方模板默认是 123456，不改就是给全网开门。

为什么不用 SQLite

能跑，但多容器并发下容易锁。有自己的库更稳，反正 postgres 也是白送一个容器。

16.4 起好之后

1. `docker compose up -d`，浏览器开 `http://你的IP:2999`
2. 首次登录默认账号 root / 123456 —— **第一件事就是改掉**
3. 进「渠道」→ 新增 → 选服务商类型（OpenAI 兼容最常用）→ 填你原来的

```
   base_url 和 key → 测试一下能不能通
4. 进「令牌」→ 新建一个 token —— 这就是你客户端要用的 sk-xxx
5. 以后客户端统一填：http://你的IP:2999/v1 + 这个 token
```

16.5 接进你的各处客户端

手机上的 RikkaHub：
  提供商 → API Base URL 填 `http://你的IP:2999/v1`（或域名）
  API Key 填刚建的令牌 → 拉模型列表，选一个 → 保存

astrbot：
  服务提供商 → 类型 OpenAI 兼容 → 同样填中转站的地址和令牌

好处立刻体现：以后换服务商，只在 new-api 后台加个渠道，
手机和 astrbot 一个字都不用改。

16.6 别把它变成公共厕所

① 别把 2999 直接晾在公网 —— 要么套 HTTPS（第 15.6 节），要么只给自己 IP 用
② 后台密码改强，能开两步验证就开
③ 给不同的用途发不同的子令牌，谁超支了单独停谁
④ 定期看「日志」页，异常调用一眼能看出来

一条经验

中转站是"你自己的基础设施"，不是"免费给朋友用"的福利社。

真给人用，一定要设额度上限。

验收

- 一个令牌能在手机 RikkaHub 上正常聊天
- 后台「日志」里能看到这次调用
- 后台默认密码已改 → 过关

下篇 · 拓展

17 拓展排错手册（对号入座）

装 Docker / 起容器

```
docker: command not found        没装好或没重登 shell
                                 → 重开一个 SSH 会话，或 usermod -aG docker $USER 后重
```

登

permission denied /var/run/...	当前用户不在 docker 组 → 同上（或用 sudo）
pull 很慢、超时	国内直连慢 → 配 daemon.json 镜像加速器，或用能通的网络
no matching manifest for arm64	镜像没有 ARM 版 → 换 x86_64 机器，或换一个有 ARM 版的镜像
port is already allocated	端口被占了 → docker ps 看谁占的；ss -tlnp | grep 端口
compose 文件报 yaml 错	多半是**缩进**（YAML 只认空格，不认 Tab） → 全用空格，层级对齐

容器起来了又挂

一直 Restarting	看日志：docker compose logs --tail=100 服务名 常见：配置写错、缺环境变量、连不上数据库
被系统杀掉（OOM）	内存不够 → free -h / dmesg | grep -i killed 加 swap，或降配（少开容器）
日志里时间不对	缺 TZ=Asia/Shanghai
中文日志乱码	容器内没有中文字体（装 t2i / 截图类插件时会遇到） 镜像里补字体包

QQ 那套不通

napcat 里 QQ 显示离线	登录态过期 → 进 6099 重新扫码；确认 ntqq 目录没有被删
astrbot 平台页一直「未连接」	三查：Token 是否一字不差 / URL 里容器名是否是 astrbot / 两个服务是否在同一个 network
napcat 日志有连接但 astrbot 收不到	反向 WS 的路径漏了 /ws
消息能收但不回	① 没带唤醒词 ② 不在白名单 ③ 模型没配或没额度 → 挨个查，日志里会写"忽略"之类的原因
群里回两遍	有两个平台适配器同时启用了 → 关掉不用的那个

微信

扫码后很快掉线	风控或协议变动 → 换小号；隔天再试；看适配器日志的具体错误码
能收不能发	有的消息类型不支持 → 看适配器日志，通常是媒体类消息的格式问题

整个号被封	用了非官方协议 → 无解。这就是为什么反复强调：**别用主号**

中转站

令牌报 401	令牌没启用 / 过期 / 额度用完
渠道报错 upstream error	渠道的 key 或 base_url 不对 → 后台点「测试」逐个验证
用量对不上	多家渠道的计费口径不同 → 以后台汇总为准，别拿服务商账单硬对

网络 / 域名

域名打不开	① DNS 没生效 ② 安全组没放 80/443 ③ Caddy 没起来 → dig 域名看解析；ufw status 看端口
证书申请失败	80 端口不通（Caddy 要先用 80 验证） → 放行 80，再 up -d
从外网访问不了，本机 curl 正常	防火墙/安全组没放行（两层都要看）

一条通用心法

分层看：**网络 → 容器 → 应用 → 配置**，一层一层往下。

先在服务器本机 curl 通，再谈外网；先看容器在不在跑，再谈应用报什么错。

90% 的"玄学问题"，都是跳过了其中一层直接猜。

验收

- 遇到问题能先说出"是哪一层"，再去动手
- 能背出 QQ 那套最难的三处（Token / 容器名 / 同网络）
- 知道"备份没验证过 = 没备份" → 过关

拓展篇到此

到这儿，你已经能把一个 AI 从"手机上的聊天框"养到"24 小时在线、在群里替我说话、还会自己记账"的东西了。

剩下的靠两样：**备份**，和**别用主号**。

AI 完全手册 · 从装到玩明白

作者：tzk（本体） · tzk

2026 版 · 命令、路径、字段名，能用复制就别手打